Tutorial Minggu 6 — Sesi 3: ETL dengan Python (Load)

Tujuan: memahami bagian Load dan **membuktikan** bahwa jalur Python menghasilkan angka yang sama dengan jalur SQL di Sesi 1.
Target: `fact_penjualan_py` = **560** baris dan omzet versi Python = **20.784.000**.
Waktu: sekitar 30–45 menit.
Bahan: `modul-pdf/05_Sesi_3_ETL_dengan_Python.pdf`, bagian Load dan Titik periksa.

Syarat: Minggu 5 selesai (kamu sudah menjalankan `etl.py`).

Langkah 1 — Jalankan ulang skrip, baca Bagian 3

```
python sesi3_etl_python/etl.py
```

Di bagian paling bawah keluaran, cari blok **BAGIAN 3 - LOAD**. Isinya kira-kira:

- jumlah baris tiap dimensi (`dim_tanggal_py`, `dim_produk_py`, `dim_cabang_py`, `dim_pelanggan_py`),
- `fact_penjualan_py`,
- omzet di tabel fakta,
- gagal lookup untuk tanggal, produk, cabang (semuanya harus 0),
- jumlah baris tanpa identitas pelanggan.

Langkah 2 — Pahami urutan Load

Buka `etl.py` dan cari tiga baris ini di bagian paling bawah:

```
dimensi = muat_dimensi(mesin_dw, mesin_sumber, bersih)   # DIMENSI DULU
fakta   = muat_fakta(mesin_dw, bersih, dimensi)          # BARU FAKTA
buat_indeks(mesin_dw)                                    # INDEKS PALING AKHIR
```

Dua aturan yang tidak bisa ditawar:

1. **Dimensi dulu, baru fakta.** Kunci dimensi baru ada setelah baris dimensinya terbentuk, sedangkan tabel fakta membutuhkan kunci itu. Membalik urutan menghasilkan tabel fakta penuh kunci kosong.
2. **Indeks paling akhir.** Memelihara indeks sambil menyisipkan banyak baris jauh lebih lambat daripada membangunnya sekali di akhir. Tabel hasil `to_sql()` lahir tanpa indeks, dan tanpa langkah `buat_indeks` query OLAP minggu depan bisa berjalan berpuluh kali lebih lambat.

Perhatikan juga konstanta `KUNCI_TIDAK_DIKETAHUI = -1`. Setiap dimensi mendapat satu baris berkunci `-1`. Itu sebabnya `dim_produk_py` berisi 9 baris (8 produk + 1 "Tidak Diketahui") dan `dim_cabang_py` berisi 4. Perannya sama dengan baris "Tanpa Pelanggan" di Sesi 2: tempat berlabuh supaya tidak ada baris fakta yang hilang hanya karena kuncinya tidak ketemu.

Langkah 3 — Cocokkan titik periksa

Pemeriksaan	Nilai benar
`fact_penjualan_py`	560 baris
Omzet di tabel fakta	20.784.000
`dim_tanggal_py`	365
`dim_produk_py` / `dim_cabang_py` / `dim_pelanggan_py`	9 / 4 / 41
Gagal lookup tanggal, produk, cabang	0, 0, 0
Baris tanpa identitas pelanggan	167
Jumlah produk berbeda	8
Tanggal gagal dibaca	0

Langkah 4 — Buktikan: dua jalur, satu angka

Dua angka pertama adalah inti seluruh sesi: **560 baris dan omzet 20.784.000, sama persis dengan Sesi 1 yang memakai SQL.** Buktikan sendiri dengan satu query:

```
mysql -u root -p -t -e "SELECT (SELECT SUM(total) FROM dw_toko.penjualan_bersih) AS
        versi_sql, (SELECT SUM(total) FROM dw_toko.fact_penjualan_py) AS versi_python;"
```

Kedua kolom harus bernilai 20784000.

Kalau berbeda, salah satu jalur melewatkan sebuah aturan. Mencari tahu aturan mana yang terlewat adalah latihan debugging terbaik di praktikum ini. Petunjuk: bandingkan jumlah baris per `sumber` di kedua jalur.

Kalau tersendat

Gejala	Sebab	Perbaikan
Gagal lookup bukan 0	kunci dimensi tidak ketemu	cek log Transform; pastikan nama produk 8 dan tanggal gagal dibaca 0
`fact_penjualan_py` bukan 560	Transform tidak menghasilkan 560	kembali ke Minggu 5, cocokkan log
`versi_sql` dan `versi_python` berbeda	salah satu jalur salah	ulangi Sesi 1 dari awal, lalu jalankan `etl.py`
Query tidak menemukan `fact_penjualan_py`	`etl.py` belum sampai Load	jalankan ulang `etl.py` sampai selesai tanpa error

Yang dikumpulkan ke LMS

1. **Screenshot:** `fact_penjualan_py` = 560 dan omzet Python = 20.784.000, boleh dari keluaran skrip atau dari query pembuktian.

2. **Jawaban:** mengapa dimensi dimuat sebelum fakta, dan mengapa indeks dibuat paling akhir? (1–3 kalimat)
3. **Opsional:** satu hal yang masih membingungkanmu.

> Sesi 3 berbobot **30%**, terbesar di praktikum ini, karena seluruh rantai Sesi 1–3 diuji di sini.

Minggu depan: Sesi 4, query OLAP.